

BikeShare Data Warehouse & Business Intelligence Solution

Table of Contents

1. Introduction	3
2. Dataset Description	3
3. Data Sources	5
4. Data Warehouse Architecture	6
4.1 Data Sources	6
4.2 ETL Layer (SSIS)	6
4.3 Staging Layer.....	6
4.4 Transformation Layer.....	6
4.5 Data Warehouse Layer	6
4.6 Reporting Layer	6
5. Data Warehouse Design	8
6. ETL Process	12
7 Analysis Queries.....	20
7.1 Total Number of Trips.....	20
7.2 Average Trip Duration.....	20
7.3 Trips by User Type.....	21
7.4 Top 10 Most Popular Start Stations	21
7.5 Trips by Month	21
7.6 Average Transaction Processing Time	22
7.7 Longest Trips	22
8 Accumulating Fact Table	23

1. Introduction

This project aims to design and implement a data warehouse solution for a bike-sharing system using real-world trip data. The primary objective is to transform raw and unstructured transactional data into a structured format that supports efficient querying, reporting, and analytical decision-making.

A dimensional modeling approach is adopted, including the creation of fact and dimension tables to enable a star schema design. The Extract, Transform, Load (ETL) process is implemented using SQL Server Integration Services (SSIS), where data is extracted from multiple sources, transformed into a consistent format, and loaded into a data warehouse.

The project also incorporates advanced data warehousing techniques such as surrogate keys, Slowly Changing Dimensions (SCD Type 2) to maintain historical data, and an accumulating fact table to track transaction lifecycle events. Overall, this implementation demonstrates how business intelligence solutions can be built to derive meaningful insights from large-scale datasets.

2. Dataset Description

The data set used in this project is based on a bike-sharing system and contains detailed trip-level information. Each record represents an individual ride and includes attributes such as ride identifiers, start and end timestamps, station identifiers, station names, and user types.

The dataset is relatively large, consisting of over one million records distributed across multiple CSV files. This volume of data provides a realistic scenario for implementing data warehousing techniques and testing ETL performance.

Key attributes in the dataset include temporal data (start and end times), categorical data (user type such as member or casual), and spatial data (station locations including latitude and longitude). These attributes allow for a wide range of analytical queries, such as trip duration analysis, station usage patterns, and user behavior trends.

Additionally, the dataset required preprocessing steps such as data type conversions, handling inconsistencies, and integrating multiple files into a unified staging area. This makes it suitable for demonstrating real-world ETL challenges and solutions in a data warehousing environment.

This dataset is suitable for data warehousing as it contains high-volume transactional data, multiple attributes for analysis, and supports the creation of fact and dimension tables. It also enables the implementation of key data warehousing concepts such as ETL processing, Slowly Changing Dimensions (SCD), and accumulating fact tables.

System Data

Where do Capital Bikeshare riders go? When do they ride? How far do they go? Which stations are most popular? What days of the week are most rides taken on? We've heard all of these questions since we launched in 2010, and we're glad to provide the data that shows you the answers from our first trip to today.

We invite developers, engineers, statisticians, artists, academics and other interested members of the public to use the data we provide for analysis, development, visualization or whatever else moves you.

This data is provided according to the [Capital Bikeshare Data License Agreement](#).

Trip History Data

Each month, we publish [downloadable files](#) of Capital Bikeshare trip data. The data includes:

- Duration – Duration of trip
- Start Date – Includes start date and time

Figure 1- Data Set

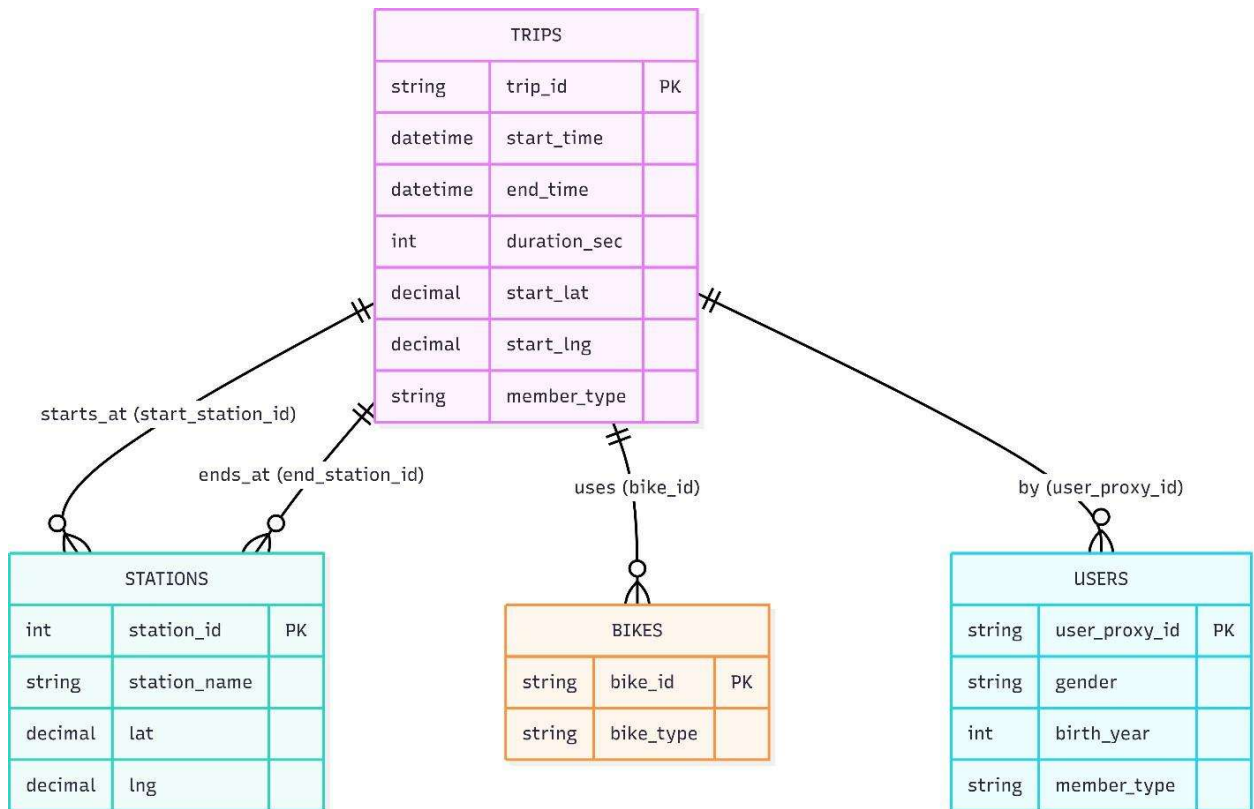


Figure 2- ER Diagram

The ER diagram represents the operational structure of the bike-sharing system. The Trip entity acts as the central transactional table, capturing each bike ride. It is associated with the User, Bike, and Station entities. Each trip is linked to one user, one bike, and two stations (start and end), forming one-to-many relationships between these entities.

3. Data Sources

The data used in this project was obtained from multiple sources in different formats.

The primary data source consists of bike trip records stored in CSV files. These files contain detailed trip-level data, including ride identifiers, timestamps, station information, and user types. The CSV format represents raw, unstructured data that requires preprocessing before loading into the data warehouse.

In addition to the primary dataset, a secondary data source was introduced in the form of a separate CSV file containing transaction completion data. This dataset includes attributes such as transaction creation time and completion time, which are used to implement the accumulating fact table.

The use of multiple data sources demonstrates the integration of heterogeneous data formats within the ETL process. All data sources were processed using SQL Server Integration Services (SSIS) and loaded into a staging database before transformation and loading into the data warehouse.

Attribute	Source Type	Source File
ride_id	CSV	trip_data.csv
started_at	CSV	trip_data.csv
ended_at	CSV	trip_data.csv
start_station_id	CSV	trip_data.csv
start_station_name	CSV	trip_data.csv
end_station_id	CSV	trip_data.csv
end_station_name	CSV	trip_data.csv
member_casual	CSV	trip_data.csv
accm_txn_complete_time	CSV	completion_data.csv

Table 1- Attribute to Source Mapping

Data enrichment was performed by deriving additional attributes from the source data. For example, trip_duration was calculated from start and end timestamps, and date attributes such as year, month, and day were extracted to support time-based analysis.

A simple hierarchy was introduced in the date dimension, allowing analysis at different levels such as year, month, and day.

4. Data Warehouse Architecture

The proposed data warehouse solution follows a layered architecture consisting of data sources, ETL processing, staging, data warehouse storage, and reporting components.

Figure 3 illustrates the overall DW/BI architecture used in this project, showing how data flows from source files through SSIS into the staging area, data warehouse, and reporting layer.

4.1 Data Sources

The system uses CSV files as the primary data source, which contain raw bike trip data. An additional CSV file is used to provide transaction completion data for the accumulating fact table. These sources represent heterogeneous data formats that need to be integrated.

4.2 ETL Layer (SSIS)

SQL Server Integration Services (SSIS) is used to perform the Extract, Transform, and Load (ETL) process. It handles reading data from source files, performing data transformations such as data type conversion and lookups, and loading data into the staging and warehouse tables.

4.3 Staging Layer

The staging database is used to temporarily store raw data before transformation. This layer ensures that the original data is preserved and allows for efficient data cleaning and preprocessing without affecting the final warehouse.

4.4 Transformation Layer

Within SSIS, various transformations are applied, including derived columns, data conversions, and lookup transformations. These processes ensure that data is consistent and aligned with the dimensional model.

4.5 Data Warehouse Layer

The processed data is loaded into the data warehouse, which consists of fact and dimension tables organized in a star schema. This layer supports efficient querying and analytical processing.

4.6 Reporting Layer

The final layer consists of SQL queries used to analyze the data and generate insights. This enables business intelligence by providing meaningful information such as trip patterns, user behavior, and system performance.

Although a separate OLAP cube was not implemented in this assignment, the reporting layer supports analytical querying through SQL-based analysis on the dimensional model.

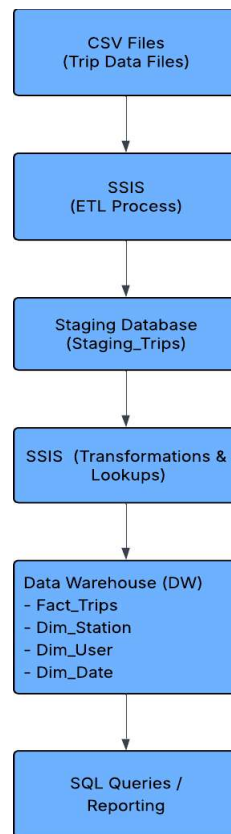


Figure 3 - High-level DW/BI architecture of the proposed solution

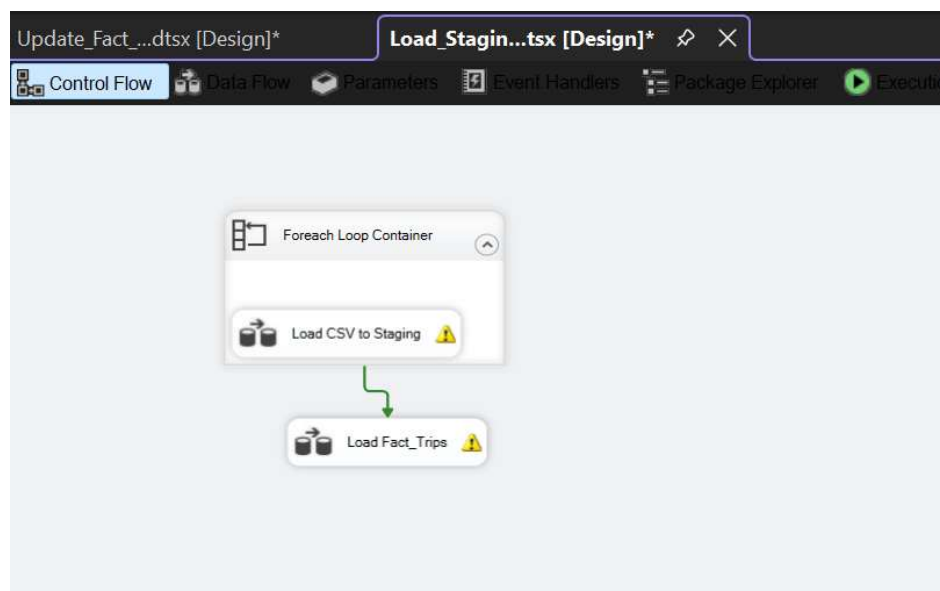


Figure 4 - SSIS Control Flow showing ETL execution sequence

Above figure shows the control flow of the SSIS package, where tasks are executed in sequence to extract, transform, and load data into the data warehouse.

5. Data Warehouse Design

The data warehouse is designed using a dimensional modeling approach, specifically a star schema. This design improves query performance and simplifies analytical reporting.

The central table in the schema is the Fact_Trips table, which stores measurable data related to bike trips, such as trip duration and transaction processing time. This table contains foreign keys that reference the dimension tables.

The dimension tables include:

1. **Dim_Station:** Stores station-related information such as station name and location. This dimension implements Slowly Changing Dimension (SCD Type 2) to maintain historical changes in station attributes.
2. **Dim_User:** Stores user type information, such as member or casual users.
3. **Dim_Date:** Stores date-related attributes such as year, month, and day to support time-based analysis.

Surrogate keys are used in all dimension tables to uniquely identify records and improve join performance. The relationships between the fact and dimension tables form a star schema.

Assumptions made during the design include:

4. Each trip is associated with a single user, date, and start/end station.
5. Station changes are tracked using SCD Type 2.
6. Date dimension is pre-populated to support efficient lookups.

Grain of the Fact Table:

Each record in Fact_Trips represents a single bike trip (one row per ride_id).

5.1 Star Schema Overview

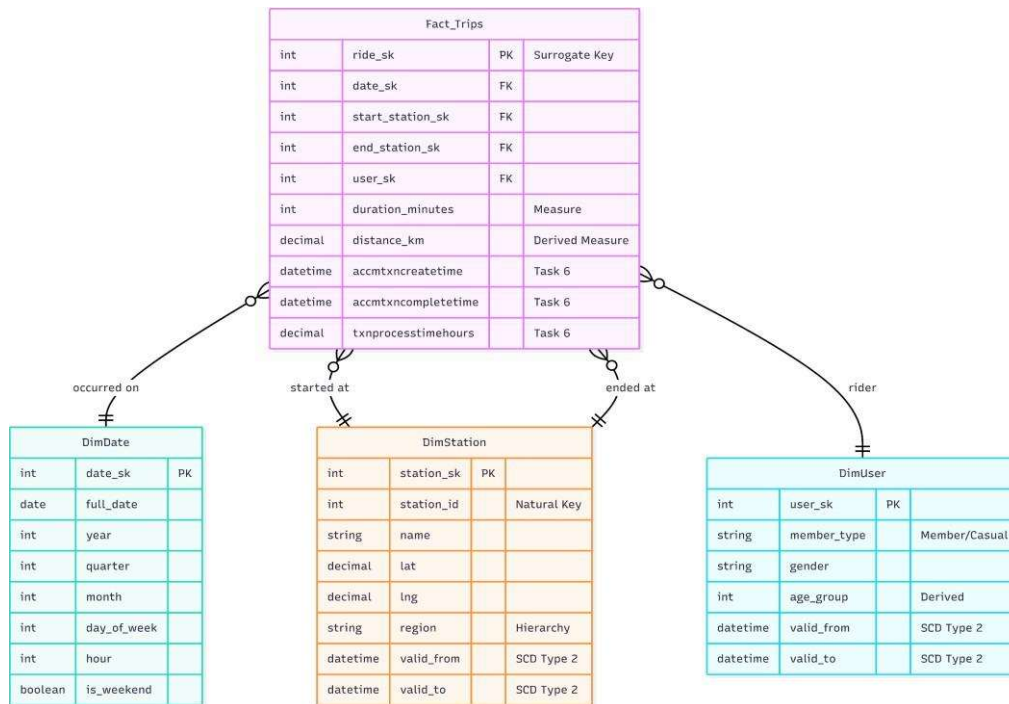


Figure 5 - Star Schema of the Data Warehouse

5.2 Dimension Tables

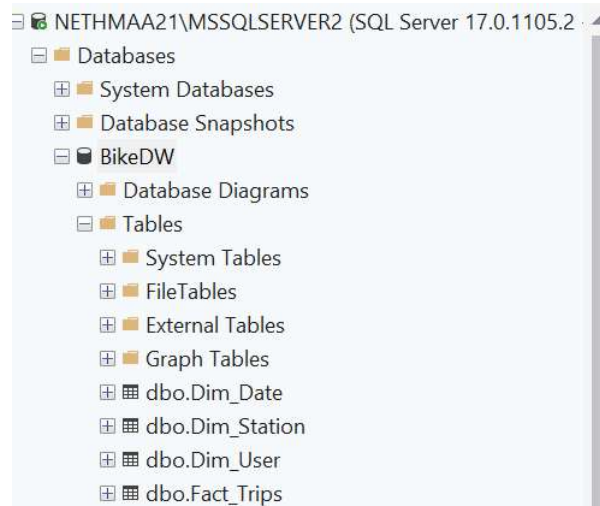


Figure 6- Dimension tables in the data warehouse

95 | SELECT TOP 10 * FROM Dim_Station;

91 % 1 0

Ln: 95, Ch: 1 SPC

Results Messages

	station_key	station_name	lat	lng	start_date	end_date	current_flag	station_id
1	1	Oklahoma Ave & D St NE	NULL	NULL	2026-03-24	NULL	1	31712
2	2	Langston Blvd & N Cleveland St	NULL	NULL	2026-03-24	NULL	1	31079
3	3	Test Updated	NULL	NULL	2026-03-24	2026-03-24	0	31905
4	4	Franklin St & S Washington St	NULL	NULL	2026-03-24	NULL	1	31907
5	5	Baltimore Ave & Van Buren St / Riverdale Park St...	NULL	NULL	2026-03-24	NULL	1	32402
6	6	Ridge Rd Community Center	NULL	NULL	2026-03-24	NULL	1	31718
7	7	Tysons One Pl & Chain Bridge Rd	NULL	NULL	2026-03-24	2026-03-24	0	32203
8	8	Oglethorpe St & 42nd Ave	NULL	NULL	2026-03-24	NULL	1	32408
9	9	4th & W St NE	NULL	NULL	2026-03-24	NULL	1	31500
10	10	Massachusetts Ave & Dupont Circle NW	NULL	NULL	2026-03-24	NULL	1	31200

Figure 7- Sample data from Dim_Station table

Above figure shows the Dim_Station table, which stores station-related information. The table implements Slowly Changing Dimension Type 2, where historical changes are tracked using start_date, end_date, and current_flag attributes.

5.3 Slowly Changing Dimension (SCD Type 2)

The Slowly Changing Dimension Type 2 logic was implemented to track changes in station attributes over time.

When a change in station name is detected for an existing station_id, the current record is expired by updating the end_date and setting current_flag to 0. A new record is then inserted with the updated values, a new start_date, and current_flag set to 1.

- SELECT station_id, COUNT(*)
FROM Dim_Station
GROUP BY station_id
HAVING COUNT(*) > 1;

104 | SELECT station_id, COUNT(*)
105 | FROM Dim_Station
106 | GROUP BY station_id
107 | HAVING COUNT(*) > 1;

91 % 1 0

Results Messages

	station_id	(No column name)
1	31051	4
2	31096	4
3	31109	4
4	31203	4
5	31220	6
6	31222	4
7	31231	4
8	31253	4
9	31627	4
10	31700	4
11	31802	4
12	31803	4
13	31905	2
14	31908	6
15	31971	6
16	32203	4

Figure 8- SCD Type 2 demonstrating multiple records per station

- SCD Logic

```

37
38
39 UPDATE Dim_Station
40 SET current_flag = 0,
41     end_date = GETDATE()
42 FROM Dim_Station d
43 JOIN BikeDW_Staging.dbo.Staging_Trips s
44 ON d.station_id = s.start_station_id
45 WHERE d.current_flag = 1
46     AND d.station_name <> s.start_station_name;
47
48 INSERT INTO Dim_Station (station_id, station_name, start_date, current_flag)
49 SELECT s.start_station_id, s.start_station_name, GETDATE(), 1
50 FROM (
51     SELECT DISTINCT start_station_id, start_station_name
52     FROM BikeDW_Staging.dbo.Staging_Trips
53 ) s
54 LEFT JOIN Dim_Station d
55 ON s.start_station_id = d.station_id
56 AND d.current_flag = 1
57 WHERE d.station_id IS NULL
58     OR d.station_name <> s.start_station_name;
59
60 SELECT station_id, station_name, start_date, end_date, current_flag
61 FROM Dim_Station
62 WHERE station_id = 31905
63 ORDER BY start_date;

```

91 % No issues found

	station_id	station_name	start_date	end_date	current_flag
1	31905	Test Updated	2026-03-24	2026-03-24	0
2	31905	Test Station Updated	2026-03-24	NULL	1

Figure 9 - SCD Logic

5.4 Fact Table

```

97 SELECT TOP 10 * FROM Fact_Trips;
98

```

91 % 1 0 Ln: 97, Ch: 1 SPC CRLF UTF-8

	trip_key	ride_id	bike_type	start_time	end_time	start_station_name	start_station_id	end_st
1	1	65F0ACD101BF0D49	classic_bike	2023-01-04 19:34:07.000	2023-01-04 19:39:29.000	East Falls Church Metro / Sycamore St & 19th St N	31904	W Colu
2	2	D75158CE73DC43F0	classic_bike	2023-01-27 15:26:38.000	2023-01-27 19:21:36.000	Carroll & Westmoreland Ave	32025	Fenton
3	3	33E8589625FF7CA	classic_bike	2023-01-05 20:44:38.000	2023-01-05 20:51:18.000	15th & L St NW	31276	Thoma
4	4	E1F055A1651F47A1	classic_bike	2023-01-03 17:45:14.000	2023-01-03 17:57:23.000	Hartland Rd & Harle Pl	32255	Merrlie
5	5	88CC90CEEC298BAF	classic_bike	2023-01-03 05:18:46.000	2023-01-03 05:25:50.000	Merrifield Cinema & Merrifield Town Center	32235	Hartlan
6	6	3E1CDC543ED2A000	classic_bike	2023-01-11 16:55:16.000	2023-01-11 17:21:54.000	Smithsonian-National Mall / Jefferson Dr & 12th ...	31248	23rd &
7	7	E03B9FCE501F5E04	classic_bike	2023-01-15 20:31:03.000	2023-01-15 20:35:51.000	23rd & M St NW	31128	23rd &
8	8	CA68C8BCC263B5DE	docked_bike	2023-01-20 23:47:23.000	2023-01-21 00:57:52.000	18th & C St NW	31284	18th &
9	9	6F1923AB4B46D1FB	classic_bike	2023-01-26 18:01:49.000	2023-01-26 18:04:20.000	22nd & P St NW	31285	24th &
10	10	5DFBEFA8A610F133	classic_bike	2023-01-28 17:47:24.000	2023-01-28 17:59:00.000	23rd & M St NW	31128	24th &

Figure 10 - Fact_Trips table with surrogate keys and measures

Above figure shows the Fact_Trips table, which contains transactional data related to bike trips. It includes foreign keys referencing dimension tables and measures such as trip duration and transaction processing time.

6. ETL Process

The ETL process is implemented using SQL Server Integration Services (SSIS) and follows a structured sequence of steps to ensure data integrity and consistency.

Data is extracted from two sources: trip data CSV files and a separate completion dataset CSV.

6.1 Extraction: Data is extracted from multiple CSV files using a Foreach Loop Container in SSIS. This allows automated processing of multiple files without manual intervention.

6.2 Loading to Staging: The extracted data is loaded into the Staging_Trips table in the staging database. This step stores raw data without transformations.

6.3 Data Transformation: Several transformations are applied, including:

- Data type conversions to ensure consistency
- Derived columns to extract date values from timestamps
- Lookup transformations to map business keys to surrogate keys in dimension tables

6.4 Loading Dimensions: Dimension tables such as Dim_User, Dim_Date, and Dim_Station are populated. The Dim_Station table uses SCD Type 2 logic to handle changes in station attributes over time.

6.5 Loading Fact Table: The Fact_Trips table is populated using the transformed data and surrogate keys obtained from lookups.

6.6 Accumulating Fact Update: A separate ETL process updates the fact table with transaction completion data, including creation time, completion time, and processing duration.

The ETL tasks are executed in a specific order, starting from data extraction and staging, followed by dimension loading, and finally fact table loading and updates.

The ETL pipeline loads data in the order: Extraction → Staging → Dimensions → Fact → Accumulating Fact Update

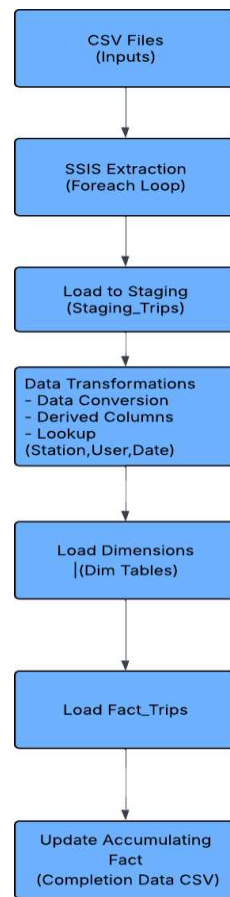


Figure 11 - ETL Process Flow Diagram

- Extraction (Multiple Sources)

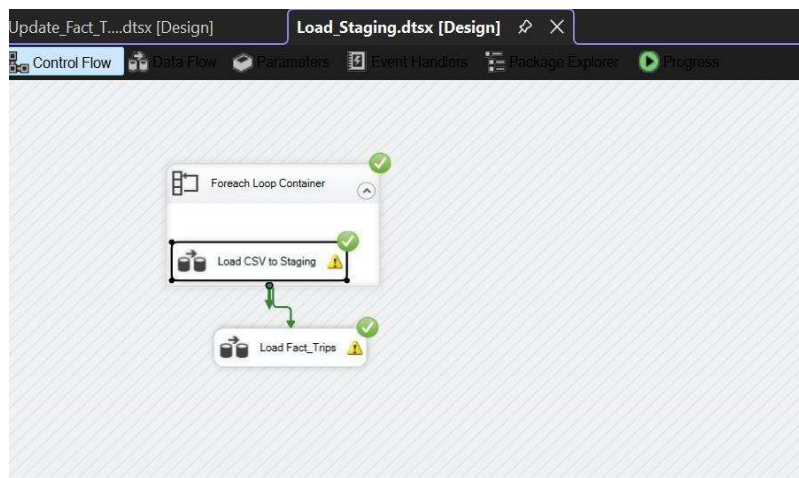


Figure 12 - Foreach Loop Container (Control Flow)

Above figure shows the use of a Foreach Loop Container in SSIS to iterate through multiple CSV files. This enables automated extraction of data without manual intervention.

- Load to Staging

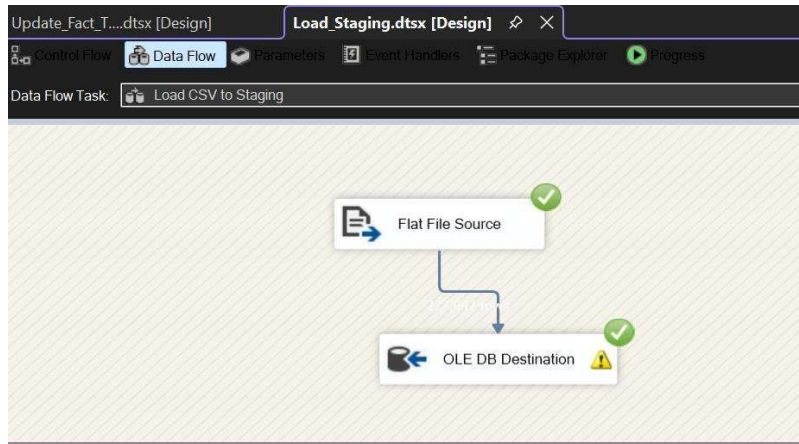


Figure 13 - Data Flow (Flat File Source → OLE DB Destination)

This data flow extracts raw data from CSV files and loads it into the Staging_Trips table. The staging layer stores unprocessed data for further transformations.

- Transformations

Data Conversion Transformation

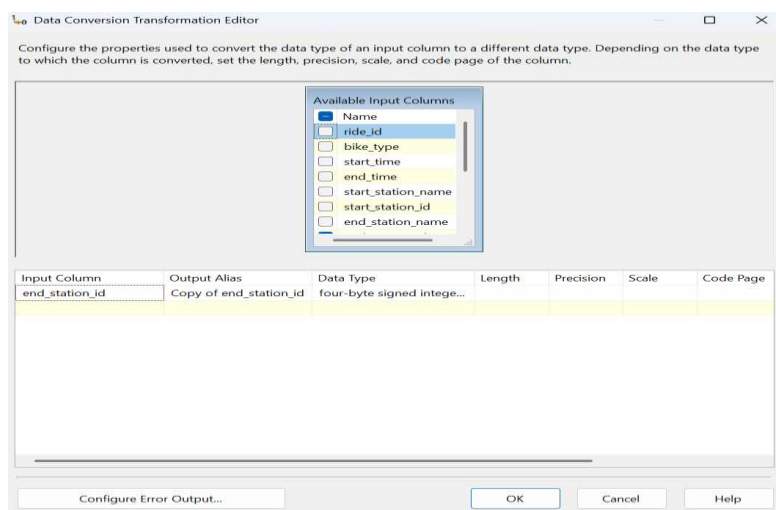


Figure 14 - Data Conversion

Above transformation converts data types (e.g., string to integer) to ensure compatibility between source data and dimension tables during lookup operations.

Lookups

■ Start Station Lookup

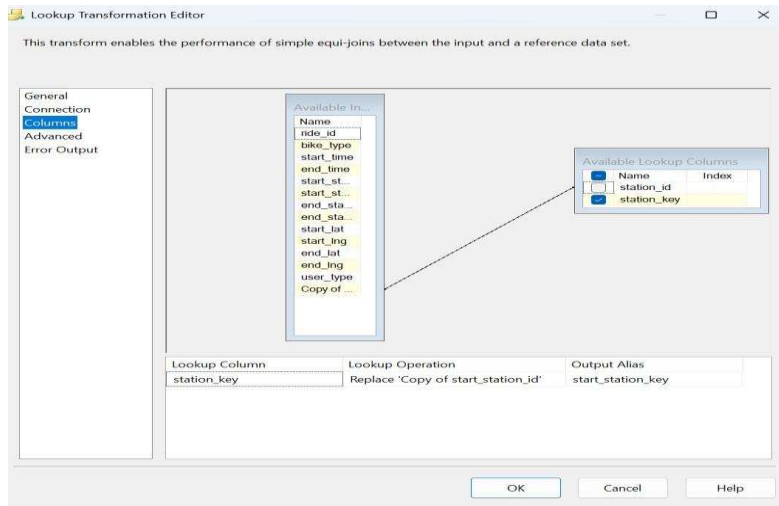


Figure 15 - Start Station Lookup (mapping)

This lookup transformation maps the business key (start_station_id) from the staging data to the surrogate key (station_key) in the Dim_Station table.

■ End Station Lookup

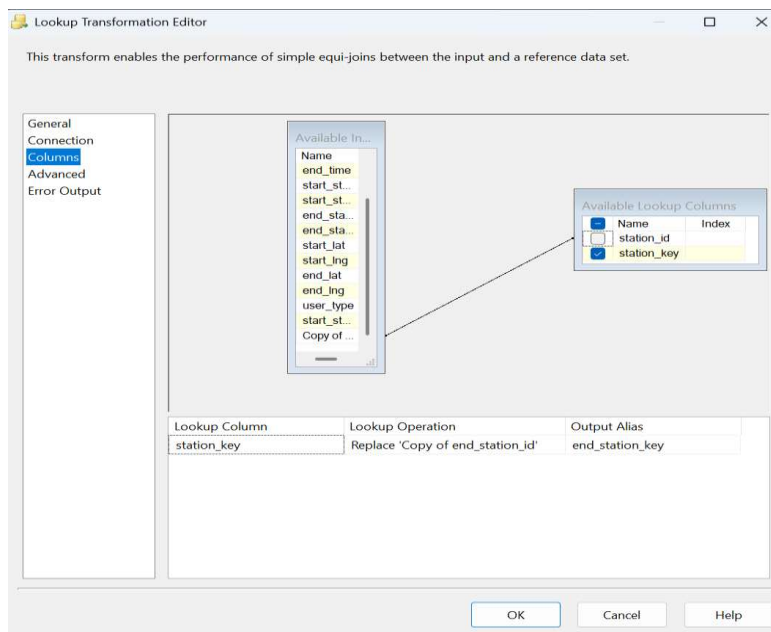


Figure 16 - End Station Lookup (mapping)

■ Date Lookup

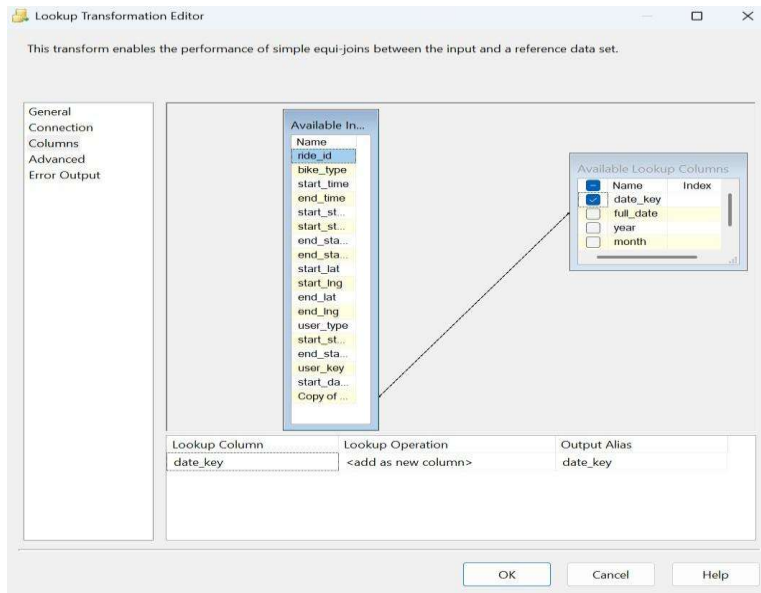


Figure 17- Date Lookup

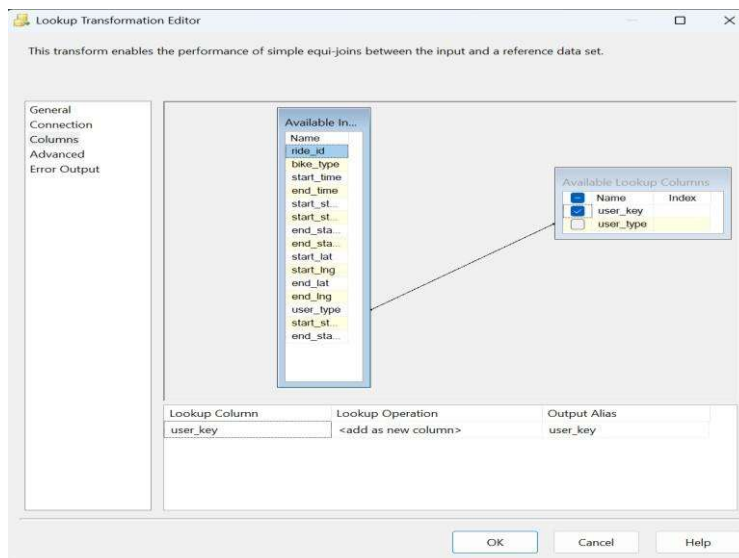


Figure 18 - User Lookup

Derived Column (Date Conversion)

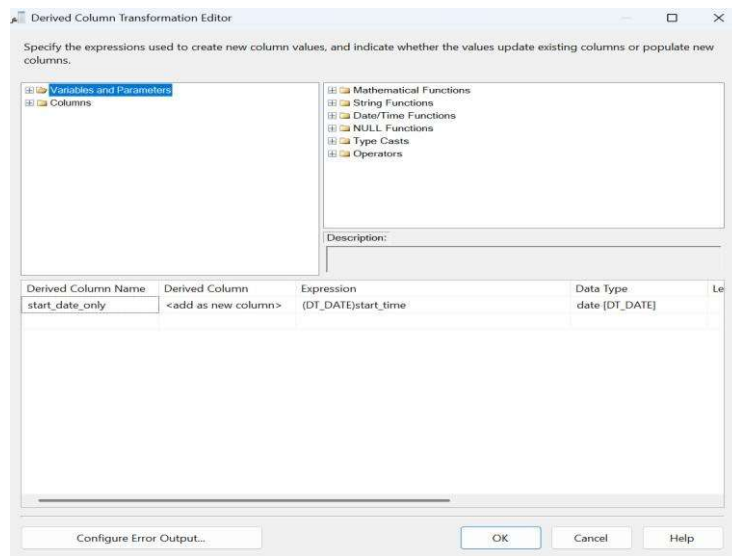
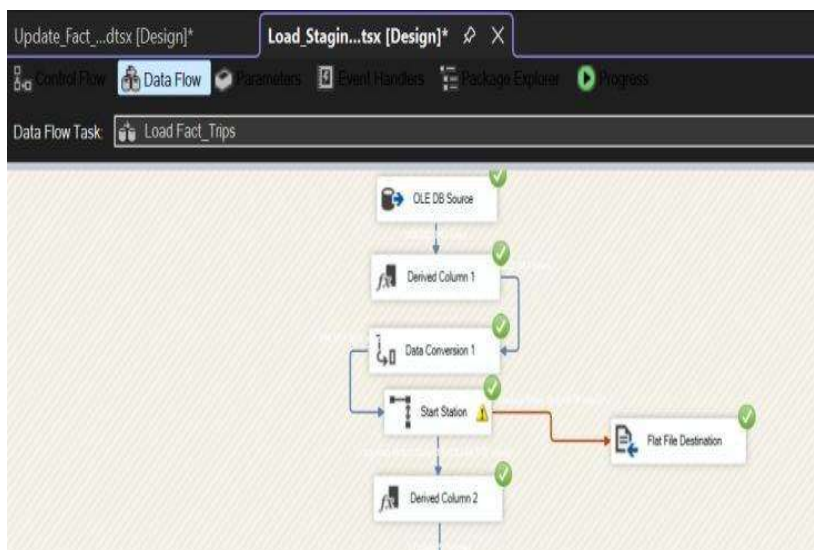
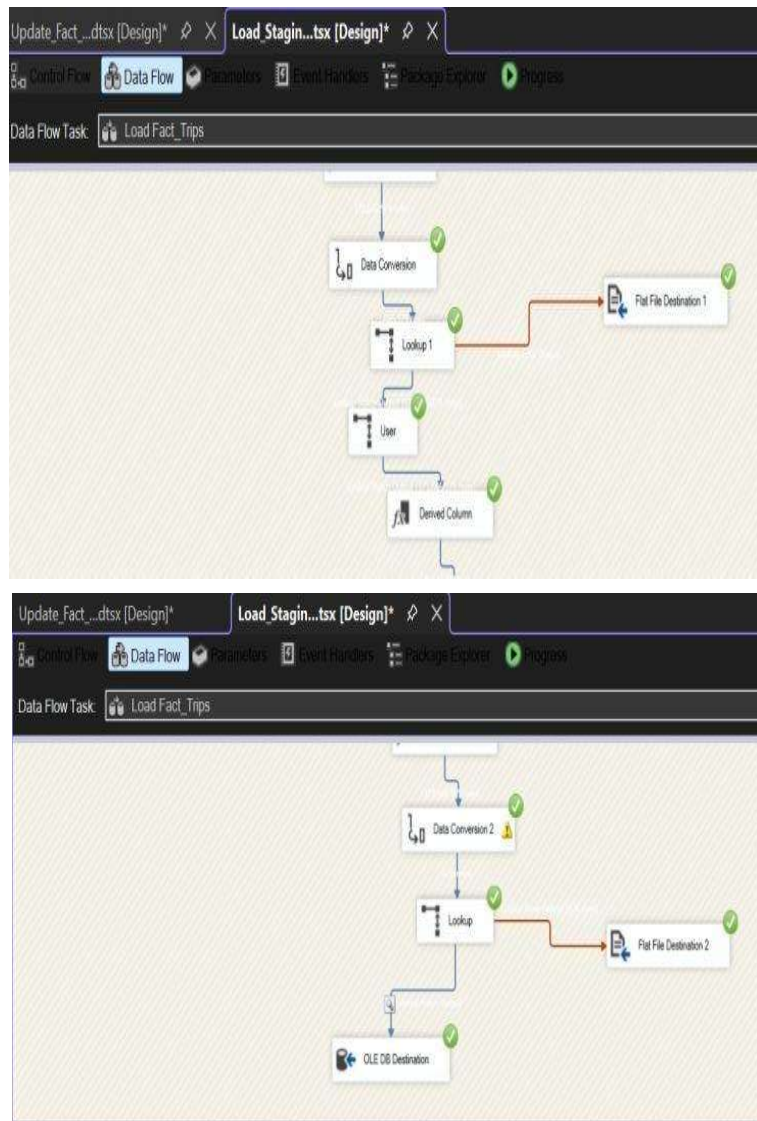


Figure 19 - Derived Couolumn

This transformation extracts the date component from the datetime field, allowing it to match with the date dimension during lookup.

- Load Dimensions





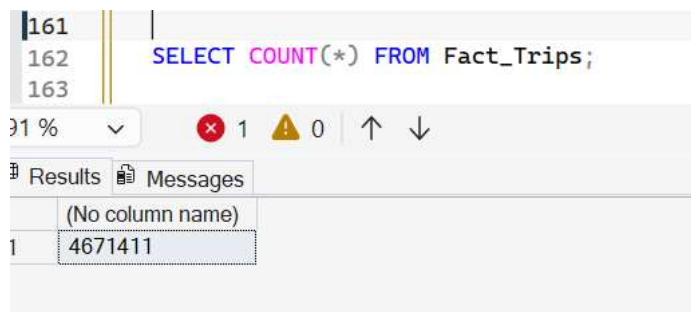
These figures show the complete ETL pipeline where transformed data, along with surrogate keys from dimension tables, is loaded into the Fact_Trips table.

6.7 Error Handling

Lookup transformations were configured to handle unmatched records appropriately to avoid null key insertions. Invalid or unmatched rows were either redirected or filtered to maintain data integrity.

6.8 Validation

1. `SELECT COUNT(*) FROM Fact_Trips;`
2. `SELECT *
FROM Fact_Trips
WHERE start_station_key IS NULL
OR end_station_key IS NULL
OR user_key IS NULL
OR date_key IS NULL;`



These queries validate that the ETL process loaded data correctly and that no foreign keys are missing.

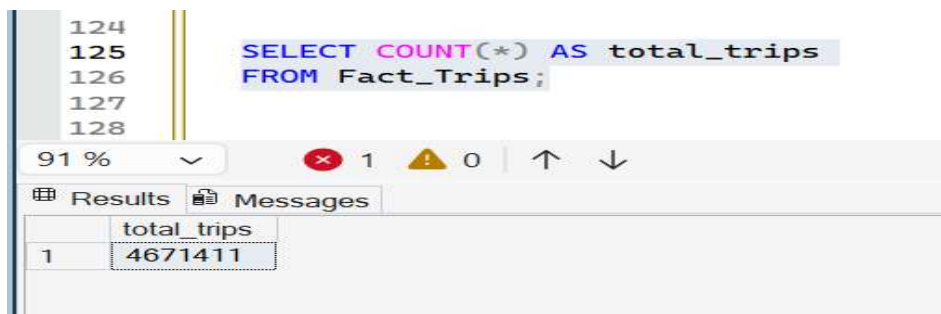
7 Analysis Queries

The data warehouse enables analytical queries to extract meaningful insights from the bike-sharing dataset. The following queries demonstrate how the data warehouse supports business intelligence and decision-making.

7.1 Total Number of Trips

This query calculates the total number of trips recorded in the system.

```
SELECT COUNT(*) AS total_trips  
FROM Fact_Trips;
```



The screenshot shows a SQL query editor with the following query:

```
SELECT COUNT(*) AS total_trips  
FROM Fact_Trips;
```

The results pane shows a single row with the column name 'total_trips' and the value 4671411.

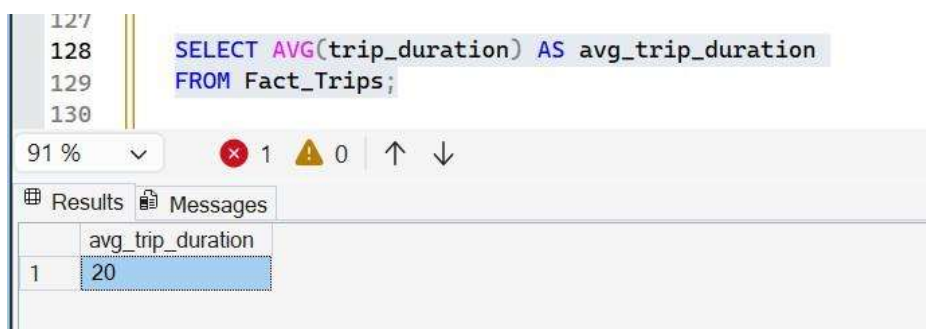
	total_trips
1	4671411

This provides an overview of the dataset size and system usage.

7.2 Average Trip Duration

This query calculates the average duration of trips.

```
SELECT AVG(trip_duration) AS avg_trip_duration  
FROM Fact_Trips;
```



The screenshot shows a SQL query editor with the following query:

```
SELECT AVG(trip_duration) AS avg_trip_duration  
FROM Fact_Trips;
```

The results pane shows a single row with the column name 'avg_trip_duration' and the value 20.

	avg_trip_duration
1	20

This helps analyze how long users typically use the serv

7.3 Trips by User Type

This query shows the distribution of trips between different user types.

```
SELECT u.user_type, COUNT(*) AS total_trips
FROM Fact_Trips f
JOIN Dim_User u ON f.user_key = u.user_key
GROUP BY u.user_type;
```

This analysis helps understand customer segmentation (e.g., member vs casual users).

7.4 Top 10 Most Popular Start Stations

This query identifies the most frequently used start stations.

```
SELECT TOP 10 s.station_name, COUNT(*) AS trip_count
FROM Fact_Trips f
JOIN Dim_Station s ON f.start_station_key = s.station_key

GROUP BY s.station_name
ORDER BY trip_count DESC;
```

This helps identify high-demand locations.

7.5 Trips by Month

This query analyzes trip trends over time.

```
SELECT d.year, d.month, COUNT(*) AS total_trips
FROM Fact_Trips f
JOIN Dim_Date d ON f.date_key = d.date_key
GROUP BY d.year, d.month
ORDER BY d.year, d.month;
```

This supports seasonal trend analysis.

7.6 Average Transaction Processing Time

This query calculates the average processing time for completed transactions.

```
SELECT AVG(txn_process_time_hours) AS avg_processing_time  
FROM Fact_Trips  
WHERE accm_txn_complete_time IS NOT NULL;
```

This helps evaluate system performance and efficiency.

7.7 Longest Trips

This query identifies trips with the highest duration.

```
SELECT TOP 10 ride_id, trip_duration  
FROM Fact_Trips  
ORDER BY trip_duration DESC;
```

This helps detect unusual or extended usage patterns.

8 Accumulating Fact Table

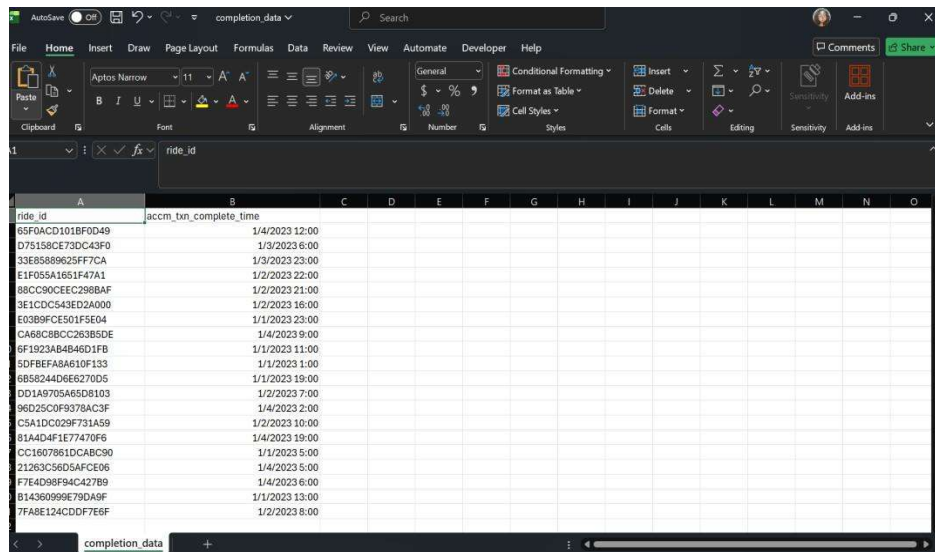
An accumulating fact table is used to track the lifecycle of a transaction by updating records as events occur over time. Unlike traditional fact tables, which are insert-only, accumulating fact tables allow updates to existing records.

In this project, the Fact_Trips table was extended with additional columns to support this concept, including accm_txn_create_time, accm_txn_complete_time, and txn_process_time_hours.

The accm_txn_create_time is set when the record is initially loaded, while the accm_txn_complete_time is updated later using a separate dataset. The processing time is calculated as the difference between these two timestamps.

A separate SSIS package was developed to read completion data from an external CSV file and update the corresponding records in the fact table. This enables tracking of transaction processing duration and supports performance analysis.

If a transaction ID from the completion dataset does not exist in the Fact_Trips table, the record is ignored during the update process to prevent invalid updates.



ride_id	accm_txn_complete_time
65F0ACD101BF0049	1/4/2023 12:00
D75158CE73DC43F0	1/3/2023 6:00
13E8589625FF7CA	1/3/2023 23:00
E1F055A1851F47A1	1/2/2023 22:00
88CC90CEEC298BAF	1/2/2023 21:00
3E1CDC543ED2A000	1/2/2023 16:00
E03B9FCE501F5E04	1/1/2023 23:00
CA68C8CC26385DE	1/4/2023 9:00
6F1923A84B46D1FB	1/1/2023 11:00
5DFBEFA8A610F133	1/1/2023 1:00
6B58244D6E6270D5	1/1/2023 19:00
DD1A9705A65D8103	1/2/2023 7:00
96D25C0F9378AC3F	1/4/2023 2:00
C5A1DC029F731A59	1/2/2023 10:00
81A4D4F1E77470F6	1/4/2023 19:00
CC1607861DCABC90	1/1/2023 5:00
21263C56D5AFCE06	1/4/2023 5:00
F7E4D98F94C427B9	1/4/2023 6:00
B14360999E79DA9F	1/1/2023 13:00
7FABE124CDDF7E6F	1/2/2023 8:00

Figure 20 - Completion dataset

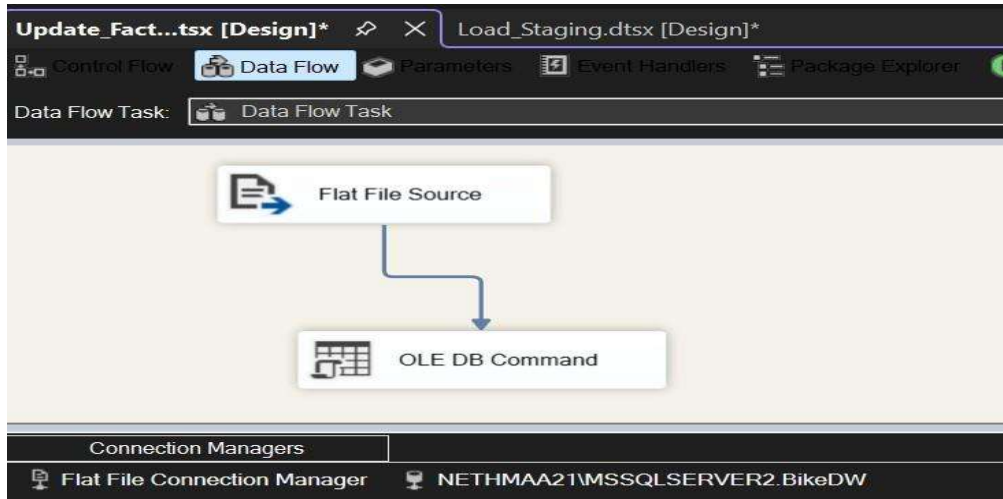


Figure 21 - SSIS package for updating fact table

8.1 Validation

```

SELECT TOP 10
    ride_id,
    accm_txn_create_time,
    accm_txn_complete_time,
    txn_process_time_hours
FROM Fact_Trips
WHERE accm_txn_complete_time IS NOT NULL;

```

113

114

115

116

117

118

119

SELECT TOP 10

ride_id,

accm_txn_create_time,

accm_txn_complete_time,

txn_process_time_hours

FROM Fact_Trips;

91 %

1

0

↑

↓

Results

Messages

	ride_id	accm_txn_create_time	accm_txn_complete_time	txn_process_time_hours
1	65F0ACD101BF0D49	2026-03-22 15:43:46.787	2023-01-04 12:00:00.000	5
2	D75158CE73DC43F0	2026-03-22 15:43:46.787	2023-01-03 06:00:00.000	5
3	33E85889625FF7CA	2026-03-22 15:43:46.787	2023-01-03 23:00:00.000	5
4	E1F055A1651F47A1	2026-03-22 15:43:46.787	2023-01-02 22:00:00.000	5
5	88CC90CEEC298BAF	2026-03-22 15:43:46.787	2023-01-02 21:00:00.000	5
6	3E1CDC543ED2A000	2026-03-22 15:43:46.787	2023-01-02 16:00:00.000	5
7	E03B9FCE501F5E04	2026-03-22 15:43:46.787	2023-01-01 23:00:00.000	5
8	CA68C8BCC263B5DE	2026-03-22 15:43:46.787	2023-01-04 09:00:00.000	5
9	6F1923AB4B46D1FB	2026-03-22 15:43:46.787	2023-01-01 11:00:00.000	5
10	5DFBEFA8A610F133	2026-03-22 15:43:46.787	2023-01-01 01:00:00.000	5

Figure 22- Updated accumulating fact table

Above query verifies that the completion time has been updated correctly and that the transaction processing time has been calculated based on the difference between creation and completion timestamps.

